Group 22 Reflection Report

Introduction

This project was divided into two distinct phases, exploring the application of Reinforcement Learning (RL) across environments of varying complexity.**Part 2** focused on the stochastic **FrozenLake 8x8** environment, emphasizing algorithmic tuning and reward engineering to overcome sparse rewards.**Part 3** addressed a complex, randomized **Cargo Collection Task**, necessitating the implementation of a **Deep Q-Network (DQN)** supported by a robust **Object-Oriented Programming (OOP)** architecture.

Part2 Reflection: Overcoming Sparce Rewards

(Frozenlake Q-learning)

In the FrozenLake 8x8 challenge, the primary difficulty was the "sparse reward" problem, where the agent receives no feedback until it reaches the goal.

A. Key Strategies and Results

According to the experimental results in frozenlake_modified.py, we achieved a success rate of approximately **70%** in the slippery setting. This was achieved through three critical optimizations:

1. **Reward Shaping:** We manually restructured the environment's feedback mechanism to guide the agent:
    - **Step Penalty (-0.001)**: This forced the agent to seek the shortest path rather than wandering aimlessly.
    - **Hole Penalty (-0.2)**: Instead of a neutral zero reward, we introduced a negative penalty for falling into holes, teaching the agent to actively avoid dangerous tiles.
    - **Destination Reward (1.5)**: We amplified the success signal to

ensure it propagated back to the starting state effectively.

2. **Dynamic Learning Rate (LR) with Success Spike:** We implemented a custom LR scheduler. While the LR generally decayed from 0.7 to 0.01 to stabilize convergence, we introduced a mechanism where the LR spiked to **0.9** whenever the agent successfully reached the goal. This simulated a "biological epiphany," ensuring successful trajectories were prioritized in memory.
3. **Correct Handling of Truncated Episodes:** We differentiated between terminated (falling/winning) and truncated (timeout) states. For timeouts, we used bootstrapping (discount_factor * max_Q) to update the value function, ensuring the agent understood the game ended due to a time limit, not a bad action.

Part 3 Reflection: OOP Architecture and Deep Learning (DQN Robot)

Moving to the randomized cargo task, the state space explosion rendered tabular Q-Learning obsolete. We adopted a DQN approach structured around solid OOP principles.

A. System Architecture Analysis (UML)

As illustrated in the **UML Class Diagram**, the system design demonstrates high cohesion and low coupling.

1. **Inheritance and Polymorphism:**
    - **Design**: We established a base Cargo class, from which GoodCargo, BadCargo, and LimitedCargo inherit.
    - **Reflection**: This design leveraged polymorphism to handle randomized object generation. The robot does not need to know the specific type of object it encounters; it simply calls the unified get_reward() interface. The object itself determines whether to

grant points, deduct points, or disappear, adhering to the **Open/Closed Principle**.

2. **Encapsulation and Separation of Concerns:**
    - **Design**: The system is split into three distinct modules: CargoGame handles core logic and rendering; CargoEnv acts as a wrapper to standardize the Gym interface; and DQNAgent manages the neural networks.
    - **Reflection**: This separation made debugging significantly easier. For instance, changing the neural network structure in DQNAgent does not affect the game logic in CargoGame. Similarly, the game state (e.g., score, robot_pos) is encapsulated within CargoGame, preventing external interference.
3. **Composition:**
    - **Design**: The DQNAgent uses composition to manage its components, containing two neural network instances (policy_net and target_net) and a ReplayBuffer.
    - **Reflection**: This structure correctly implements the **Double DQN** architecture. By maintaining a separate Target Network, we stabilized the training process and mitigated the overestimation bias inherent in standard Q-Learning.

B. Evolution from Q-Table to DQN

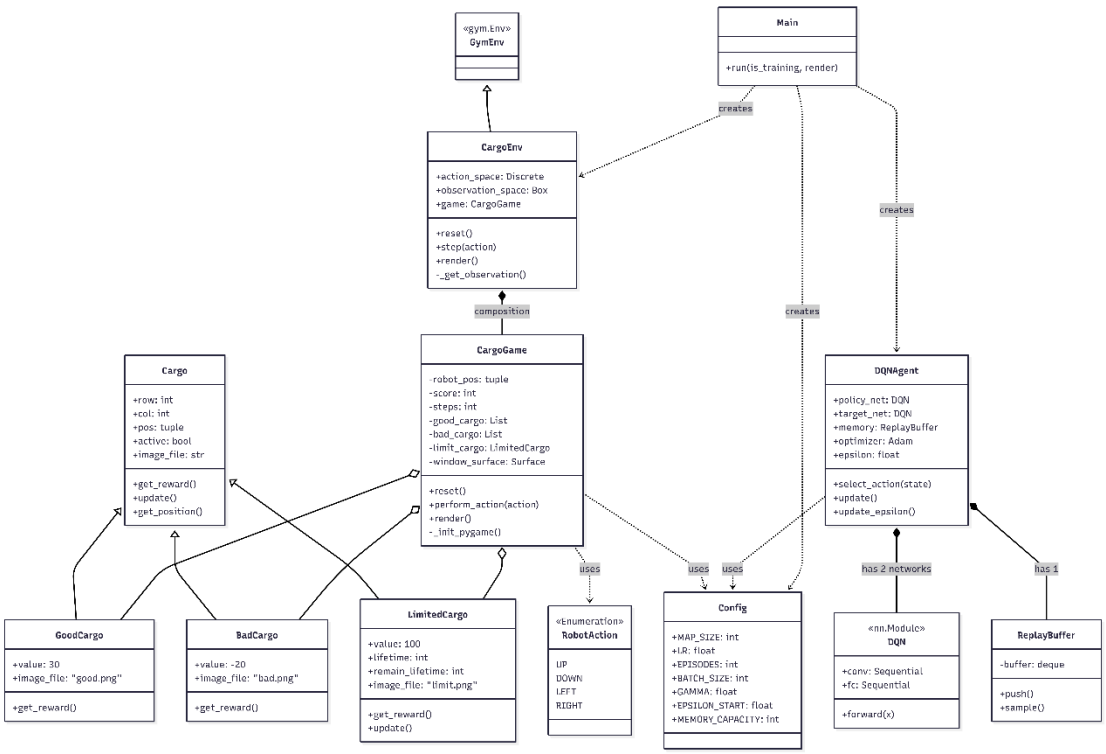
The transition from Part 2 to Part 3 represented a shift from **memorization to generalization**.

- **Part 2**: The Q-Table simply memorized the value of specific coordinates (x, y).
- **Part 3**: The DQN processes the entire map via the observation_space. The neural network learns **visual features** and relative spatial relationships (e.g., "moving towards 'G' is positive"), allowing the agent to perform well even on maps with randomized cargo positions.

Conclusion

This project demonstrated a progression from foundational to advanced Reinforcement Learning implementation. In **Part 2**, we learned to push the limits of traditional algorithms through **Hyperparameter Tuning** and **Reward Engineering**. In **Part 3**, we integrated **Software Engineering (OOP)** to manage system complexity and applied **Deep Learning** to solve problems with large, stochastic state spaces.

The UML Diagram of part3



Our github website：

https://github.com/Kevin-trytry/Group-Project-main